Agent Postmortem Generator

System Architecture Document

Master reference for module boundaries, data flow, and extension strategy

Document purpose: define the production architecture for the MVP and establish the rules that future modules, adapters, and outputs must obey. This document intentionally assumes a Python monorepo and a local-first execution model.

Scope: one generic JSON trace format for MVP ingestion, canonical normalization, deterministic analysis, and report generation. Future adapters are extension points, not part of the core architecture.

Non-goals: microservices, distributed orchestration, prompt-chained probabilistic diagnosis, and any architecture that exists mostly to impress people who do not have to maintain it.

1. Executive Overview

Agent Postmortem Generator is a local-first developer tool that converts failed AI agent execution traces into structured, blameless postmortem reports. The system is designed for engineers who need to understand what failed, why it failed, how the failure propagated, and what to fix first.

The architectural center of gravity is the canonical trace model. Every input format, including future framework adapters, must normalize into the same internal representation before analysis begins. This keeps the core deterministic, testable, and framework independent.

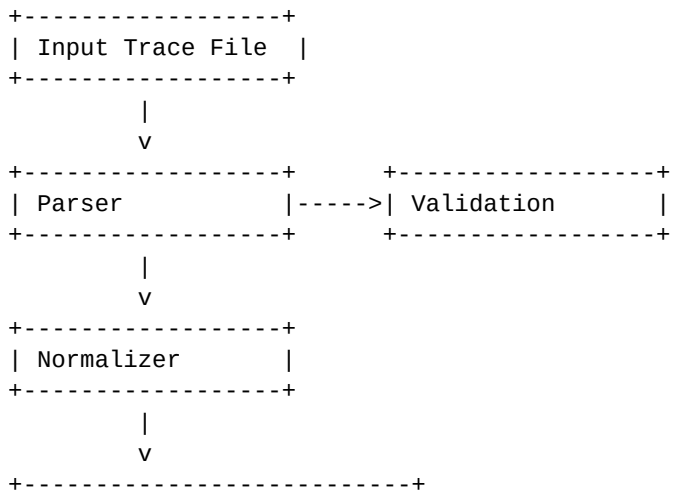
The system is deliberately layered so that parsing, normalization, analysis, and rendering remain separable. The result should feel like an infrastructure tool, not a notebook experiment with a CLI glued onto it.

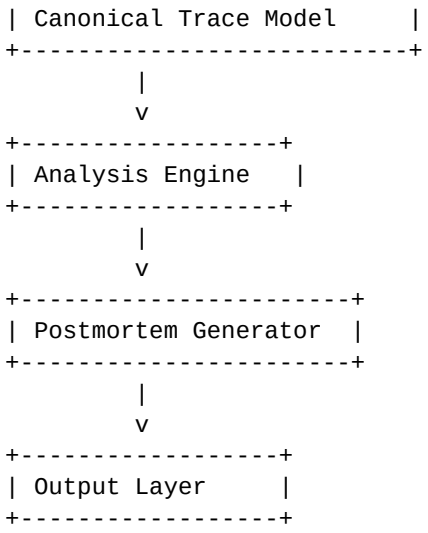
2. Overall System Goals

Goal	Implication for architecture
Local-first	No mandatory server, no network dependency in the MVP, and no hidden remote state.
Deterministic	Same input trace must produce the same canonical trace and the same report output.
Framework agnostic	Framework-specific logic ends at adapter boundaries.
Testable	Each stage must be independently unit-tested and contract-tested.
Maintainable	Low coupling, explicit dependencies, and small modules with one reason to change.
Open-source friendly	Repository structure, docs, and contribution flow must be obvious to outsiders.

3. High-Level Architecture

The system is organized as a linear pipeline with narrow, well-defined stages. Only the adapter layer knows about trace-source peculiarities. The rest of the system operates on the canonical trace model.





4. Complete Processing Pipeline

Stage	Responsibility	Output
1. Input	Read a generic JSON execution trace from disk or stdin.	Bytes or text
2. Parse	Convert raw JSON into a typed intermediate structure.	Parsed raw trace
3. Validate	Check schema, required fields, and structural sanity.	Validated raw trace
4. Normalize	Map raw trace into canonical trace model.	Canonical trace
5. Analyze	Reconstruct causality and identify failure modes.	Analysis findings
6. Generate	Assemble a structured postmortem narrative.	Postmortem document model
7. Render	Emit Markdown, Mermaid, JSON, or future PDF.	Final artifact

The important thing here is that the analysis stage never consumes raw framework logs directly. Raw input is a liability; canonical data is the contract. This saves the project from slowly mutating into a compatibility swamp.

5. Major Modules

Module	Primary responsibility
cli	Command-line entrypoint, argument parsing, execution wiring.
io	Input loading, file handling, and output path resolution.
parser	JSON parsing and raw schema validation.
adapters	Framework-specific translators into the canonical model.

normalizer	Canonicalization, event ordering, identifier normalization.
core	Canonical trace model and domain entities.
analysis	Causality reconstruction, failure classification, summarization.
generator	Postmortem assembly and section composition.
renderers	Markdown, Mermaid, JSON, future PDF output.
reporting	Formatting helpers and presentation utilities.
tests	Contract, fixture, regression, and deterministic behavior tests.

6. Responsibilities of Each Module

Module	What it is allowed to do
Parser	Accepts only well-formed generic JSON input. Rejects malformed structure early with precise errors.
Normalizer	Transforms parsed input into canonical entities, assigns stable identifiers, orders events, and fills derived fields only when derivation is deterministic.
Canonical Model	Defines the vocabulary of the project: traces, runs, spans, events, tool calls, failures, and causal links.
Analysis Engine	Runs deterministic heuristics against canonical data to infer sequence, blast radius, root causes, and contributing factors.
Postmortem Generator	Turns findings into an engineering report with standardized sections and an evidence trail.
Output Layer	Serializes the postmortem into formats without changing meaning or analysis.

7. Data Flow Between Modules

```
Raw trace
   |
   v
Parser -> ParsedRawTrace
   |
   v
Validator -> ValidatedRawTrace
   |
   v
Normalizer -> CanonicalTrace
   |
   v
Analysis Engine -> AnalysisResult
   |
   v
```

```
Postmortem Generator -> PostmortemDocument
   |
   v
Renderers -> Markdown / Mermaid / JSON / PDF
```

Data must flow forward only. No downstream stage should mutate upstream data structures in place. If a stage needs different shape or enrichment, it returns a new object. That is how you keep analysis reproducible instead of turning it into a haunted house of side effects.

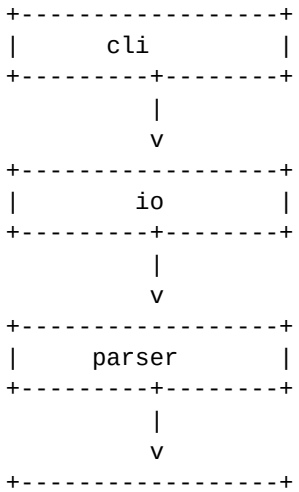
8. Canonical Trace Model Overview

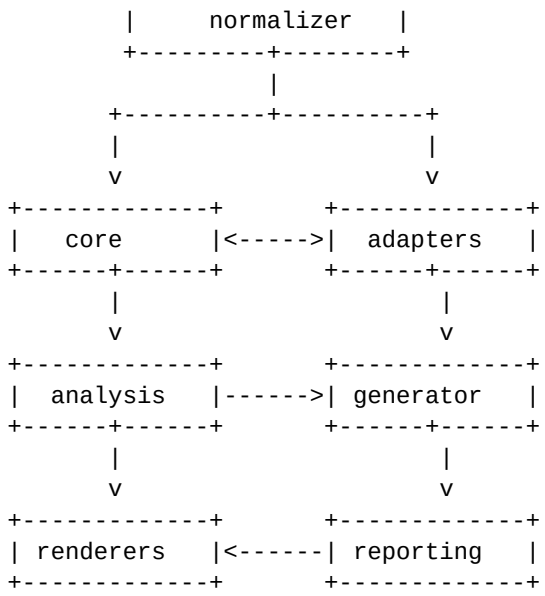
The canonical trace model is the internal language of the project. It should be stable across adapters and expressive enough to represent the information required for causality analysis without carrying framework-specific noise.

Entity	Meaning
TraceRun	A single execution instance with metadata, timestamps, and status.
TraceNode	A unit of work such as a step, tool call, agent action, or sub-run.
TraceEvent	A timestamped state change or significant milestone.
TraceEdge	A relationship between nodes, usually parent-child or dependency.
FailureSignal	The primary failure evidence extracted from the trace.
DerivedFinding	Analysis output with confidence and supporting evidence.

The model should distinguish observed facts from derived inferences. Observations come from traces. Inferences come from analysis. Mixing the two is how tools start lying while sounding confident, a favorite human hobby.

9. Internal Dependency Graph





The core package sits at the center and must not import outward-facing modules like renderers or CLI code. That rule prevents architecture drift and keeps business logic reusable in tests and future integrations.

10. Layered Architecture

Layer	Contents	Rule
Interface layer	CLI, future API wrappers, batch entrypoints	Thin orchestration only
Application layer	Pipeline coordination, request lifecycle	Calls services and maps errors
Domain layer	Canonical trace model and analysis logic	No framework dependencies
Infrastructure layer	File I/O, adapters, serialization	Replaceable implementation details

11. Core Domain Objects

Object	Why it exists
RunMetadata	Source, framework, timestamps, identifiers, version info.
NodeIdentity	Stable ID, parent link, kind, and role.
EventRecord	State transitions, timestamps, payload, and severity.
FailureEvidence	Error messages, exceptions, termination signals, timeouts.
CausalityLink	Observed or inferred relationship with a confidence level.
PostmortemSection	Structured output section with evidence references.

12. Communication Between Components

Communication is synchronous and in-process for the MVP. That is not a limitation; that is a deliberate defense against needless complexity. Components communicate by typed objects, not by stringly typed blobs and wishful thinking.

From -> To	Payload
Parser -> Normalizer	Parsed raw structure with validation context.
Normalizer -> Analysis	Canonical trace and normalization notes.
Analysis -> Generator	Findings, evidence, and confidence.
Generator -> Renderers	Document model ready for serialization.

13. Error Propagation Strategy

Error class	Propagation rule
Input errors	Fail fast with line- or field-specific messages.
Validation errors	Aggregate multiple schema issues when safe to do so.
Normalization errors	Explain which canonical invariant could not be satisfied.
Analysis errors	Surface incomplete evidence rather than invent conclusions.
Rendering errors	Keep analysis result intact and fail only the final emission step.

Errors must preserve provenance. A user should be able to tell whether the issue came from the source trace, normalization logic, or an internal defect. Hand-waving error messages are just operational debt wearing a fake mustache.

14. Extension Points

Extension point	Constraint
Input adapters	Add support for new frameworks without changing canonical analysis.
Analyzers	Add specialized deterministic analyses behind well-defined interfaces.
Report sections	Add new postmortem sections without rewriting the pipeline.
Renderers	Add PDF or other outputs without changing the document model.

15. Adapter Architecture

Adapters are boundary translators. Their job is to convert framework-specific traces into the canonical model and nothing else. Adapters are not allowed to embed analysis policy, ranking logic, or narrative wording.

```
Framework trace
      |
      v
Adapter
      |
      v
Canonical trace normalizer
      |
      v
Shared analysis pipeline
```

Adapter	Role
LangGraph adapter	Maps graph nodes, edges, tool calls, and run metadata.
CrewAI adapter	Maps agent tasks, tool usage, and collaboration events.
OpenAI Agents SDK adapter	Maps run steps, tool invocations, and outcomes.
AutoGen adapter	Maps conversations, agent actions, and dependency links.
Generic JSON adapter	MVP input path; defines the baseline schema.

16. Output Architecture

Output	Use case
Markdown	Primary human-readable format. Stable and diff-friendly.
Mermaid	Useful for execution graphs and causality diagrams.
JSON	Machine-readable artifact for automation and future integrations.
PDF	Deferred output for polished reporting and sharing.

The output model should be generated from the same postmortem document object, not rebuilt separately per renderer. That keeps every format semantically aligned instead of subtly disagreeing like a committee in a trench coat.

17. Folder Structure Recommendations

```
agent-postmortem-generator/
  pyproject.toml
  README.md
  LICENSE
  docs/
```

```
  architecture/
  principles/
  roadmap/
  examples/
src/
  apm/
    cli/
    io/
    parsing/
    normalization/
    core/
    analysis/
    generator/
    renderers/
    adapters/
    reporting/
    errors/
tests/
  unit/
  contract/
  fixtures/
  regression/
examples/
scripts/
pyproject.toml
```

Keep source under a single top-level package, with tests and documentation separated cleanly. Avoid flattening everything into a giant src folder full of mood swings.

18. Recommended Python Package Organization

Package	Responsibility
apm.cli	Entrypoint and command orchestration.
apm.io	File and stream handling.
apm.parsing	JSON parsing and raw validation.
apm.normalization	Canonical model conversion.
apm.core	Domain objects and invariants.
apm.analysis	Deterministic failure analysis.
apm.generator	Postmortem assembly.
apm.renderers	Output format serializers.
apm.adapters	Framework-specific ingestion layers.
apm.errors	Typed error definitions and mapping.

19. Future Scalability Considerations

- Keep the core pipeline single-process until there is a proven need to split it.
- If workload grows, scale by optimizing input size, fixture reuse, and incremental analysis before adding distributed infrastructure.
- Treat PDF export, interactive graphs, and web UIs as outputs or wrappers, not reasons to compromise the core model.

- Preserve deterministic behavior even when adding optional LLM-assisted refinement in future versions.

Scalability here means sustaining maintainability as the codebase grows, not prematurely inventing infrastructure because the diagram looked lonely.

20. Security & Privacy Considerations

Concern	Architectural requirement
Local-first default	Avoid sending traces anywhere unless the user explicitly requests it.
Sensitive data handling	Assume traces may contain tokens, secrets, customer data, or internal URLs.
Redaction readiness	Design outputs so future redaction can be added without surgery.
Least privilege	Any future integrations must operate with minimal access and explicit opt-in.

Because traces may include secrets, security cannot be an afterthought. The architecture should make it easy to keep everything local, inspectable, and removable. Convenient leakage is still leakage.

21. Risks & Architectural Tradeoffs

Risk	Mitigation
Over-normalization	Too much canonical abstraction can hide useful source detail. Preserve raw references where needed.
Under-normalization	If adapters leak framework concepts into core analysis, the model becomes brittle. Enforce invariants.
Heuristic drift	Analysis logic may become inconsistent over time. Lock it behind fixtures and golden outputs.
Renderer divergence	Different output formats can disagree. Generate all formats from one document model.

22. Architecture Decision Summary

Decision	Why it was chosen
Single-process pipeline	Chosen for MVP simplicity, determinism, and local-first behavior.
Canonical trace model	Chosen to isolate framework churn from core analysis.
Typed internal objects	Chosen to improve testability and contract clarity.
Deterministic analysis	Chosen to maximize trust and reproducibility.
Multiple renderers from one doc model	Chosen to prevent semantic drift across outputs.

Appendix: Architectural Rules That Never Change

- Raw input is never the analysis substrate.
- Adapters translate, they do not interpret.
- Inference must be distinguishable from evidence.
- Every module owns one clear responsibility.
- Determinism takes priority over cleverness.
- If a new feature violates these rules, the feature is wrong, not the rules.